basics/04/spec/models/project_spec.rb
```
it "marks a project done if its tasks are done" do
  project.tasks << task
  task.mark_completed
  expect(project).to be_done
end
```

You make the test pass by adding logic to your project's done? method:

basics/04/app/models/project.rb
```
class Project
  attr_accessor :tasks

  def initialize
    @tasks = []
  end

  def done?
    tasks.all?(&:complete?)
  end
end
```

I can't think of an easy way to break the done? method as it currently stands, so it is … well, done.

Adding Some Math

Moving on. Next, you need to be able to calculate how much of a project is remaining and the rate of completion, and then put them together to determine a projected end date.

The next test is for the project to be able to calculate how much work is remaining.

I like to take a moment before I write a test to think about what the test needs. The typical test structure has three parts:

- Given: What data does the test need? *This test needs a project, at least one complete task, and at least one incomplete task.*
- When: What action is taking place? *I'm calculating the remaining work.*
- Then: What behavior do I need to specify? *The work-calculation result.*

I also like to think about what could make the happy-path test fail; that's more helpful once you get the happy-path test in place and then start looking at it critically.

With that thought exercise over, you write your spec. Actually, there are two assertions and I've split them up. In practice I'd write them one at a time, but I don't think you need to go through all those steps explicitly. To add this to